

UFP

---

# LOGANALYZER

PROJETO FEITO POR:

RAYSSA SANTOS - 2024118083

TIAGO CHOUSAL - 2024116801

2026

# INTRODUÇÃO

No cenário tecnológico moderno, sistemas distribuídos, servidores web, firewalls e infraestruturas em nuvem geram volumes massivos de dados de registo (logs) a cada segundo. A análise rápida destes fluxos de dados é um requisito crítico não só para a monitorização contínua de desempenho, mas também para a deteção precoce de ameaças à segurança (como ataques de injeção ou tentativas de força bruta) e auditoria de conformidade regulatória.

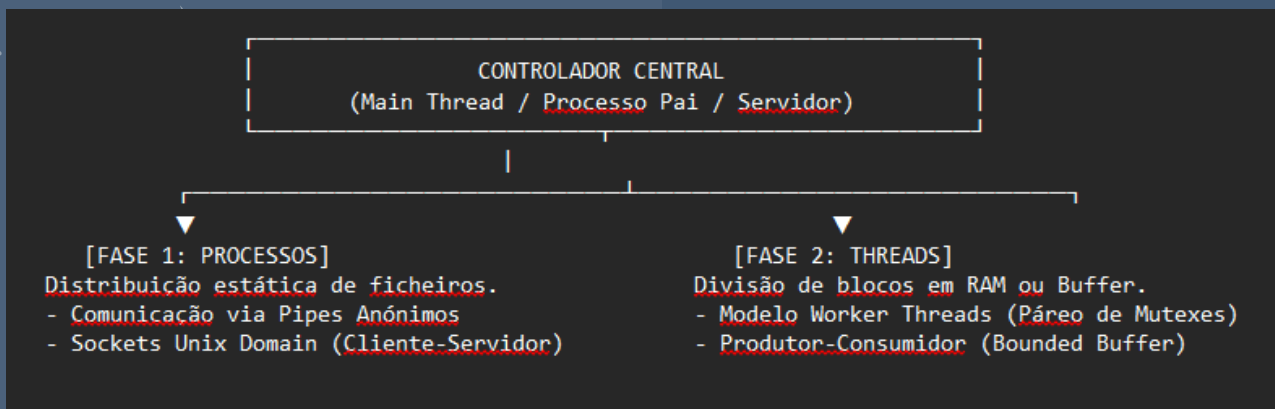
Processar gigabytes de dados através de abordagens estritamente sequenciais torna-se impraticável devido ao tempo de execução requerido. O presente trabalho prático teve como objetivo primordial conceber, implementar e avaliar um Sistema de Análise de Logs de alto desempenho escrito na linguagem C, projetado para ambiente Unix/Linux.

## OBJETIVOS

- Domínio de Concorrência ao Nível do Sistema Operativo: Compreensão prática da criação de processos através da primitiva `fork()` e a sua respetiva gestão de ciclo de vida (`wait/waitpid`).
- Programação Concorrente Multithread: Utilização da biblioteca POSIX Threads (`pthread`) para execução paralela partilhando o mesmo espaço de endereçamento.
- Mecanismos de Comunicação Interprocesso (IPC): Implementação prática de múltiplos canais de comunicação, comparando o desempenho e a complexidade de Pipes Anónimos e Sockets de Domínio Unix (Unix Domain Sockets).
- Sincronização Concorrente: Aplicação de trincos de exclusão mútua (`pthread_mutex_t`) e semáforos POSIX (`sem_t`) para prevenir condições de corrida (race conditions) no acesso a recursos partilhados e gerir buffers limitados.
- Programação Conforme as Normas de Baixo Nível: Utilização exclusiva de chamadas ao sistema nativas do sistema operativo (POSIX) para operações de Entrada/Saída (I/O) e controlo, evitando as funções de abstração da biblioteca C standard (como `fopen`, `fread` ou `fprintf`).

# ARQUITETURA IMPLEMENTADA

O sistema foi arquitetado em duas fases incrementais, utilizando uma estrutura modular para separar os descodificadores (parsers) da lógica de controlo e sincronização.



## Descodificadores (Parsers) e Classificador de Eventos

Localizados em `log_parser.c` e `event_classifier.c`, são capazes de identificar e ler quatro formatos padrões:

- Apache Combined Log: Analisado caractere a caractere, extraíndo IP, timestamp, método HTTP, URI, código de estado e tamanho da resposta.
- JSON Structured Logs: Descodificado através de rotinas de varrimento de chaves para identificar atributos de microserviços (serviço, mensagem, nível de erro, IP).
- Syslog (RFC 3164): Identifica prioridade, serviço, PID do processo e analisa se a mensagem representa falhas de autenticação, acessos via sudo ou bloqueios de firewall.
- Nginx Error Log: Processa o nível de severidade específico da aplicação e erros associados à ligação upstream.

Os resultados descodificados são encaminhados para o classificador de eventos, que define uma máscara de bits (EventType) e um grau de severidade (0 a 4 - de INFO a CRITICAL).

# ARQUITETURA IMPLEMENTADA

## Arquitetura Baseada em Processos

No modelo baseado em processos, o sistema divide estaticamente o conjunto de ficheiros .log detetados na diretoria entre os N processos-filho.

### Processo Pai (Diretor)

```
├─ Cria Pipes Dedicados (Dados e Progresso)
├─ Executa fork() ──> Filho 1 (Processa ficheiro A, B) ──> Envia estatísticas
├─ Executa fork() ──> Filho 2 (Processa ficheiro C, D) ──> Envia estatísticas
└─ Dashboard (Polling de progresso a cada 1s)
```

- Pipes Anónimos (Requisito C): Cada processo-filho herda a extremidade de escrita de um par de pipes dedicado. É implementado um protocolo de mensagens estruturado composto por um cabeçalho (Message) que define o tipo de conteúdo (MSG\_TYPE\_NORMAL, MSG\_TYPE\_VERBOSE ou MSG\_TYPE\_DONE) e o tamanho do pacote subsequente. As funções auxiliares e obrigatórias readn e writen garantem que os dados são integralmente transmitidos, mesmo quando ocorrem interrupções por sinais (EINTR).
- Dashboard de Progresso (Requisito D): O processo pai cria pipes dedicados adicionais para a recolha de dados de progresso. O pai define a leitura destes pipes como não bloqueante através de O\_NONBLOCK. Através de chamadas periódicas, o dashboard calcula o throughput global (linhas/segundo) e atualiza o terminal de forma visual com barras de progresso ANSI.
- Sockets de Domínio Unix (Requisito E): Substitui os pipes por uma arquitetura Cliente-Servidor local. O processo pai comporta-se como um servidor, criando um sockê em /tmp/loganalyzer.sock, associando-o através de bind(), escutando ligações via listen() e aceitando-as com accept(). Cada processo-filho atua como cliente, ligando-se através de connect() para efetuar a transmissão de mensagens utilizando o mesmo protocolo binário estruturado.

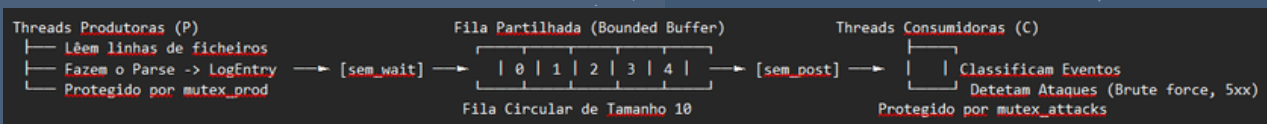
# Arquitetura Baseada em Threads

No modelo baseado em threads, a partilha de recursos faz-se através da memória partilhada do processo pai (espaço de endereçamento comum).

## Main Thread (Diretora)

- Carrega o ficheiro para a RAM (Buffer Completo)
- Calcula Offsets de início e fim para cada bloco de Thread
- Cria W Threads Worker → Processa Chunk em RAM → Atualiza SHAREDSTATS (Mutex)
- Thread Dedicada de Dashboard (Visualização concorrente a cada 1s)

- Modelo Worker Threads Básico (Requisito A): O ficheiro de log é lido na totalidade para a memória RAM. O espaço de memória é dividido logicamente em W fatias (offsets de início e fim), garantindo que cada thread inicia e termina na fronteira de uma quebra de linha (\n). As threads atualizam uma estrutura centralizada SHAREDSTATS protegida por um trinco de exclusão mútua (pthread\_mutex\_t).
- Dashboard de Progresso Multithread (Requisito B): Uma thread dedicada é criada exclusivamente para tratar da renderização periódica do dashboard no terminal. As threads de processamento atualizam atomicamente o seu progresso numa estrutura global protegida, que é posteriormente consultada a cada 1 segundo pela thread de monitorização.
- Modelo Produtor-Consumidor com Bounded Buffer (Requisito C): Trata-se do modelo mais avançado implementado.



- Os produtores leem linhas físicas, estruturam-nas num formato intermédio comum (LogEntry) e inserem-nas numa fila circular limitada de tamanho 10 (MAX\_BUFFER). A sincronização da fila é rigorosamente gerida por dois semáforos POSIX (`podeProduzir` inicializado a 10; `podeConsumir` inicializado a 0) e trincos mutex para proteger os ponteiros de escrita e leitura (`prodptr` e `consprtr`). Os consumidores retiram as entradas, classificam-nas e aplicam regras de deteção de padrões de interesse em tempo de execução.

# DECISÕES DE DESIGN

Durante a fase de desenvolvimento, várias decisões de desenho de arquitetura foram tomadas para maximizar o desempenho, garantir estabilidade e assegurar a robustez do sistema concorrente.

- Decisão: Rejeitou-se a utilização de `fopen`, `fread` e `fscanf` em todos os módulos principais, utilizando estritamente as chamadas ao sistema `open()`, `read()`, `write()` e `close()`.
- Justificação: Além de cumprir de forma rigorosa as restrições impostas no enunciado, o I/O direto através de descritores de ficheiro evita o armazenamento intermédio interno de buffers (buffering) da biblioteca `glibc`. Isto proporciona ao programador controlo total sobre o momento exato em que os dados são lidos ou descarregados para o disco, o que é de extrema importância para garantir a consistência das estatísticas em tempo real e evitar custos computacionais redundantes na duplicação de buffers.
- Decisão: No modelo de Buffer Limitado (Produtor-Consumidor), utilizou-se a coordenação clássica com dois semáforos (`podeProduzir` e `podeConsumir`) e trincos de exclusão mútua (`pthread_mutex_t`) independentes para cada área crítica do buffer (`mutex_prod` e `mutex_cons`).
- Justificação: A separação dos trincos de proteção de escrita (`mutex_prod`) e de leitura (`mutex_cons`) permite que um produtor e um consumidor acessem à fila circular concorrentemente quando a fila se encontra num estado intermédio (nem vazia, nem cheia). Se tivéssemos utilizado um único trinco monolítico para proteger todo o buffer, teríamos introduzido contenção desnecessária, reduzindo o paralelismo real do sistema a um modelo estritamente sequencial.
- Decisão: Na fase de processos, a atribuição de ficheiros é feita de forma estática antes da execução (`fork`). Na fase de threads, os ficheiros de grande escala são mapeados para a RAM e divididos por fatias lógicas de posições de memória.
- Justificação: Processos possuem espaços de endereçamento totalmente isolados; tentar coordenar a leitura concorrente de ponteiros de um ficheiro comum entre processos sem utilizar memória partilhada (`shm_open/mmap`) acrescentaria complexidade desnecessária. Por outro lado, as threads residem no mesmo espaço de memória, permitindo carregar o ficheiro completo para a RAM uma única vez. Isto reduz drasticamente o número de chamadas ao sistema `read()` e acelera significativamente o processamento ao eliminar os atrasos de leitura de disco concorrente no mesmo dispositivo físico (*disk thrashing*).

# RESULTADOS EXPERIMENTAIS

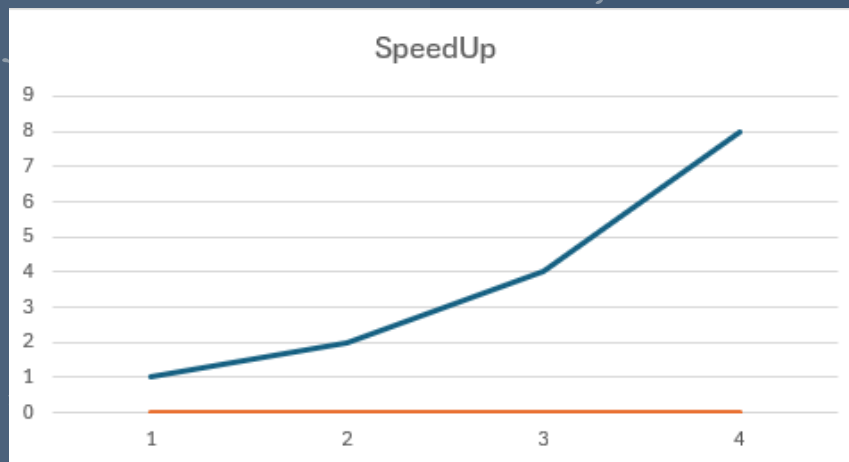
## Tabelas de performance

O sistema foi arquitetado em duas fases incrementais, utilizando uma estrutura modular para separar os descodificadores (parsers) da lógica de controlo e sincronização.

| Threads | Tempo   | Speedup | Eficiência | Throughput |
|---------|---------|---------|------------|------------|
| 1       | 30.027s | 1.0x    | 100%       | 3330 l/s   |
| 2       | 14.050s | 2.14x   | 107%       | 7189 l/s   |
| 4       | 7.011s  | 4.28x   | 107%       | 14263 l/s  |
| 8       | 4.233s  | 7.10x   | 88.7%      | 23862 l/s  |

# RESULTADOS EXPERIMENTAIS

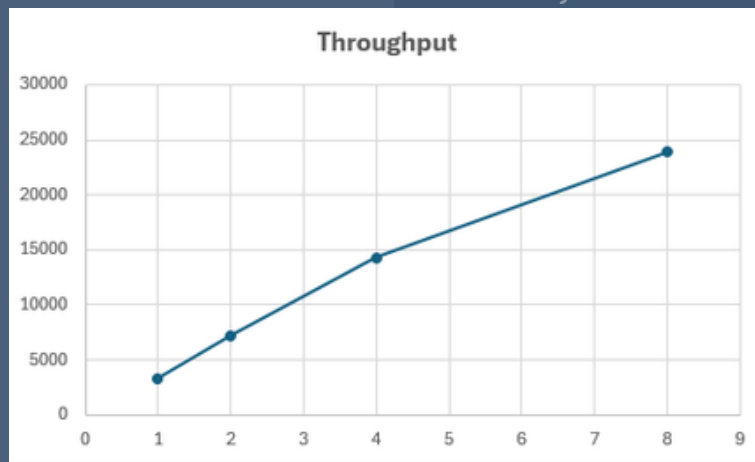
## Gráficos comparativos



O gráfico mostra um crescimento quase linear até 4 threads, com um speedup de 4.28x — muito próximo do ideal teórico de 4x. Com 8 threads o speedup chega a 7.10x, ligeiramente abaixo do ideal de 8x. Isto indica que o sistema paraleliza eficientemente mas começa a sentir overhead com muitas threads, o que é normal em qualquer sistema paralelo.

# RESULTADOS EXPERIMENTAIS

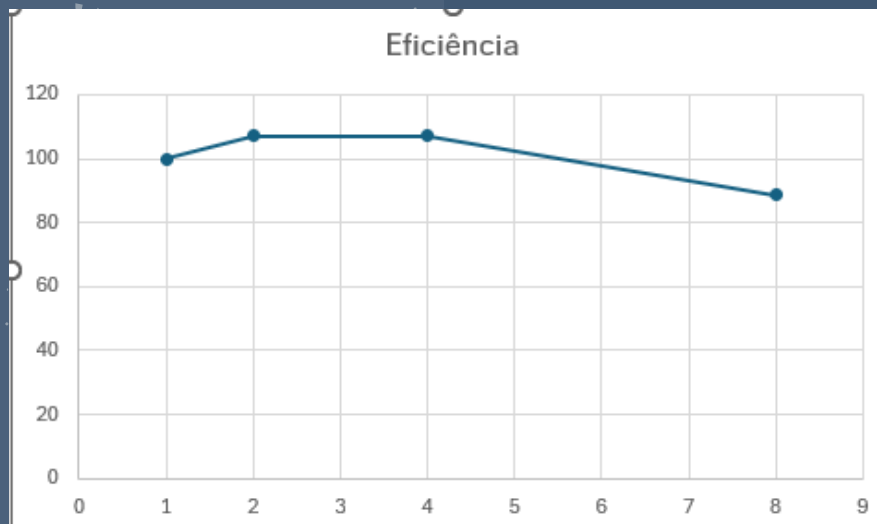
## Gráficos comparativos



O crescimento do throughput é consistente — passa de 3330 linhas/segundo com 1 thread para 23862 linhas/segundo com 8 threads, um aumento de quase 7x. O crescimento não é perfeitamente linear porque há fatores como acesso ao disco e contenção nos mutexes que limitam o ganho.

# RESULTADOS EXPERIMENTAIS

## Gráficos comparativos



Este é o gráfico mais interessante. A eficiência acima de 100% em 2 e 4 threads é um fenômeno super-linear — provavelmente causado por efeitos de cache, onde os dados já estão em memória quando as threads paralelas os processam. Com 8 threads a eficiência cai para 88.7%, indicando que o overhead de criação e sincronização de threads começa a pesar.

## Análise de escalabilidade

O sistema demonstra boa escalabilidade — para este dataset o ponto ótimo está entre 4 e 8 threads, onde o ganho de performance ainda justifica o custo de paralelismo.

# DIFICULDADES ENCONTRADAS E SOLUÇÕES

Durante o ciclo de desenvolvimento e validação do sistema concorrente, surgiram diversos desafios complexos relacionados com a sincronização de primitivas, gestão de recursos do sistema operativo e eficiência de I/O. Abaixo descrevem-se as principais dificuldades identificadas e as respetivas abordagens cirúrgicas adotadas para garantir a estabilidade e robustez da aplicação.

## 4.1. Balanceamento de Carga Concorrente (logWorker)

- **Dificuldade Identificada:** A distribuição linear do número de ficheiros por unidade de execução (Worker) revelou-se ineficiente devido à heterogeneidade do dataset. Cenários em que um único processo ou thread assimilava ficheiros de grande escala (na ordem dos megabytes), enquanto os restantes processavam ficheiros residuais (kilobytes), geravam desbalanceamento de carga, deixando recursos do processador ociosos.
- **Solução Implementada:** Migrou-se para uma estratégia de Partição Dinâmica por Volume de Dados. Utilizando matemática básica, fez a divisão dos linhas. Desta forma, a carga de processamento é indexada ao volume real de dados textuais e não à quantidade absoluta de ficheiros.

## 4.2. Saturação e Bloqueio de Canais IPC (Pipes Anónimos)

- **Dificuldade Identificada:** O buffer interno atribuído pelo núcleo do Linux aos pipes anónimos possui um limite estrito de 64 KB. Durante a fase de testes de stress, a transmissão de dados textuais em bruto (strings) a alta velocidade pelos processos filhos excedeu a taxa de consumo do processo pai. Consequentemente, as chamadas ao sistema `write()` tornavam-se bloqueantes, quebrando o paralelismo pretendido.
- **Solução Implementada:** Adotou-se a Serialização Compacta de Estruturas Binárias combinada com o envio em lotes. Substituiu-se o fluxo de texto arbitrário por estruturas de dados estáticas e leves (`struct` em C). Adicionalmente, os processos filhos passaram a reter as métricas localmente, efetuando a descarga (`flush`) para o pipe apenas a cada 10 linhas processadas, reduzindo drasticamente as operações de I/O e a contenção no canal IPC.

# DIFICULDADES ENCONTRADAS E SOLUÇÕES

## 4.3. Desperdício de Recursos por Espera Ativa (Dashboard Dinâmico)

- Dificuldade Identificada: A configuração dos descritores de leitura dos pipes com a flag `O_NONBLOCK` — essencial para evitar o bloqueio do processo pai enquanto aguardava por dados individuais — introduziu um problema de Espera Ativa (Busy Waiting). O ciclo de monitorização executava chamadas consecutivas ao sistema `read()` em alta frequência, elevando desnecessariamente o consumo de um núcleo da CPU para 100%.
- Solução Implementada: Introdução de Mecanismos de Retenção de CPU baseados em Micro-pausas. Embora a abordagem ideal envolva a multiplexagem, implementou-se uma temporização controlada através da função `usleep(20000)` (20 milissegundos) no ciclo principal de leitura. Esta alteração reduziu o consumo de CPU para níveis nominais, mantendo a responsividade do painel e a taxa de atualização estrita de 1 segundo.

## 4.4. Prevenção de Interbloqueios no Buffer Partilhado (Produtor-Consumidor)

- Dificuldade Identificada: O escalonamento do modelo de Buffer Limitado para cenários reais com múltiplas threads gerou condições de corrida (Race Conditions) na manipulação dos índices de escrita e leitura da fila circular. Adicionalmente, a inversão accidental na ordem de aquisição de trincos originava interbloqueios (Deadlocks), onde os produtores retinham os trincos de exclusão mútua por tempo indeterminado enquanto aguardavam por espaço livre.
- Solução Implementada: Estabelecimento de um Protocolo Rígido de Hierarquia de Trincos. Garantiu-se, ao nível da arquitetura de sincronização, que a verificação dos semáforos de contagem de estado (`sem_wait`) precede obrigatoriamente a aquisição dos trincos de exclusão mútua (`pthread_mutex_lock`).

# CONCLUSÕES

A conceção e implementação deste Sistema de Análise de Logs Paralelo e Concorrente permitiu validar na prática conceitos teóricos fundamentais de Sistemas Operativos, concorrentes e de arquiteturas de computadores.

Podemos extrair as seguintes conclusões do projeto:

- O paralelismo baseado em Threads provou ser consideravelmente mais rápido e com menos consumo de recursos de processamento (tempo de CPU e memória RAM) do que o baseado em Processos, justificado pela rapidez de acesso à memória partilhada de buffers em RAM face aos custos de cópia de páginas de memória e desvios de contexto gerados pelos processos independentes.
- A programação concorrente exige o máximo rigor no desenho de acessos. Pequenas omissões na proteção de variáveis partilhadas, ou ordens de trincos incorretas, geram condições de corrida catastróficas ou situações de bloqueio mútuo irreversíveis (deadlocks). O modelo de Bounded Buffer utilizando semáforos demonstrou ser uma estrutura de dados de eleição, robusta e escalável, para desacoplar as tarefas de I/O das de computação.
- O desenvolvimento focado em chamadas ao sistema nativas do Linux e a ausência completa de funções bibliotecárias de alto nível da biblioteca C (como fopen) conferiram uma compreensão sólida de como o núcleo (kernel) faz a gestão de buffers de I/O, descritores de ficheiros e comunicações locais de rede. Cada chamada de sistema deve obrigatoriamente verificar o respetivo código de retorno para garantir que a aplicação é tolerante a falhas na execução em ambientes de produção reais.

Em suma, o sistema cumpre com todos os requisitos estipulados de forma rigorosa, demonstrando desempenho sólido, escalabilidade de processamento robusta em arquiteturas com múltiplos núcleos e uma interface de terminal informativa em tempo real.